

Software Design Specification

1. Purpose

This document specifies the internal design of PixelGrid. It focuses on modules, responsibilities, data structures, validation, configuration, error handling, maintainability, and security considerations.

2. Design scope

Included in scope:

- Tetris firmware in `Games/Tetris`.
- Breakout firmware in `Games/Breakout`.
- Shared PixelGrid library in `libraries/PixelGridcore`.
- Host-side tests in `tests`.
- CI configuration in `.github/workflows/tetris-tests.yml`.

Excluded from scope because source is not present:

- External score API backend.
- Persistent database implementation.
- Physical PCB design files, except for tutorial references and circuit PDF.

3. Module inventory

Area	File or folder	Responsibility
Tetris entry point	<code>Games/Tetris/Tetris.ino</code>	Arduino setup/loop, runtime mode selection, standalone and host orchestration, game-over submission flow.
Tetris game logic	<code>Games/Tetris/Game.h</code>	Board state, pieces, validation, movement, line clear, scoring, level timing, hold, ghost piece, rendering hooks.
Tetris input	<code>Games/Tetris/Input.h</code>	Button abstraction, debounce, edge detection, input state mapping, joystick repeat.
Tetris rendering	<code>Games/Tetris/Render.h</code>	Matrix and LCD rendering helpers, colours, text/segment output, HUD drawing.
Tetris pins/config	<code>Games/Tetris/Pins.h</code>	Hardware pins, grid dimensions, debounce and repeat timing constants.

Area	File or folder	Responsibility
Tetris host runtime	<code>Games/Tetris/HostRuntime.cpp</code> , <code>HostRuntime.h</code>	Serial frame parsing and host-rendered display support.
Tetris network submission	<code>Games/Tetris/NetSubmit.h</code>	Wi-Fi setup, NTP, HMAC signing, JSON POST to <code>/api/codes</code> , response code extraction.
Breakout entry point	<code>Games/Breakout/Breakout.ino</code>	Arduino setup/loop, frame timing, game update orchestration.
Breakout game logic	<code>Games/Breakout/Game.cpp</code> , <code>Game.h</code>	Paddle, ball, bricks, score, speed, brick drop, game-over state.
Breakout input	<code>Games/Breakout/Input.cpp</code> , <code>Input.h</code>	Button and joystick input update/latch/repeat behaviour.
Breakout rendering	<code>Games/Breakout/Render.cpp</code> , <code>Render.h</code>	Matrix drawing for paddle, ball, bricks, and game-over/score states.
Shared grid	<code>libraries/PixelGridcore/src/Pixel_Grid.h</code>	Logical grid-to-LED mapping and pixel buffer management.
Shared LCD	<code>libraries/PixelGridcore/src/LCD_Digit.h</code> , <code>LCD_Panel.h</code>	Seven-segment digit rendering and panel composition.
Tests	<code>tests/tetris_game_tests.cpp</code> , <code>tests/stubs</code>	Host-side tests and Arduino/renderer stubs.

4. Internal design: Tetris

4.1 Core data model

`TetrisGame` stores:

- `board[PLAY_H][W]`: 20 by 10 logical playfield.
- `curPiece` and `nextPiece`: active and preview tetromino state.
- `curX`, `curY`: active piece position.
- `holdType` and `holdLocked`: hold-piece state and one-hold-per-drop rule.
- `gameOver`, `score`, `totalLinesCleared`, `level`, `fallDelayMs`, `lastScoreSpeedStep`, and `tFall`.
- `PIECE_COLORS`, `PREVIEW_BG`, `PLAY_BG`, and `GHOST_COLOR` initialised from the renderer.

Tetromino definitions are stored as 4x4 bit masks in program memory with four rotations per piece.

4.2 Tetris validation and business logic

Important validation and rule functions include:

- `validAtParams`: rejects placements outside the board, below the bottom boundary, or colliding with occupied cells.
- `validAt`: validates the current piece at a candidate position and rotation.

- `clearLines`: detects full rows, shifts rows down, and returns the number of cleared lines.
- `classicLineClearScore`: calculates score using classic Tetris-style line-clear values multiplied by level plus one.
- `updateLevelOnCleared`: increments level based on total cleared lines and adjusts fall delay.
- `applyLineClearScoreAndLevel`: combines scoring, level updates, per-line speedup, and score-based speed steps.
- `lockPiece`: writes the current piece into the board and handles game-over conditions for locked blocks above the playfield.
- `doHold`: enforces hold locking until the next piece is spawned.
- `tryRotateTo`, `rotateRight`, and `rotateLeft`: implement rotation with simple horizontal wall kicks.
- `tryMove`: moves pieces if valid or locks them on failed downward movement.
- `currentFallDelay`: reduces fall delay while soft drop is held.
- `computeGhostY`: projects the current piece downward to draw a ghost position.

4.3 Tetris rendering design

Rendering is separated from game-rule decisions. `TetrisGame::render` draws locked blocks, ghost position, and current piece through the `Renderer`. `Renderer` then writes to `Pixel_Grid` and `LCD_Panel` abstractions. This supports hardware output while keeping the board rules mostly testable.

4.4 Tetris runtime states

`Games/Tetris/Tetris.ino` contains state transitions for title display, game play, game over, host mode, score submission, and fallback between host and standalone modes. The design uses loop-driven state checks rather than an explicit state-machine class.

5. Internal design: Breakout

Breakout uses a C-style module split:

- Global game state is declared in `Game.h` and defined in `Game.cpp`.
- `Breakout.ino` controls frame timing and calls input, game, and render functions.
- Bricks are represented in `bricksGrid[BRICK_H][W]` with colour values.
- Ball and paddle positions use small integer coordinates.
- Rules include paddle movement bounds, serving, ball stepping, collision against walls/bricks/paddle, score increments, ball speedup after brick hits, periodic brick row drops, and game-over checks.

This design is simple for Arduino sketches and keeps each concern in separate files.

6. Shared PixelGrid library design

6.1 `Pixel_Grid`

`Pixel_Grid` accepts an `Adafruit_NeoPixel` pointer, start index, row count, and column count. It builds a conversion table that maps logical grid cells to physical strip indexes for a serpentine layout. It provides methods to clear the grid, set the entire grid colour, set individual cell colours, read buffered cell colours, and render the pixel buffer to the strip.

6.2 `LCD_Digit`

`LCD_Digit` renders numeric characters or raw segment masks to seven LED segments. It supports per-digit on/off colours and segment-mask mode for icons or custom symbols.

6.3 `LCD_Panel`

`LCD_Panel` owns up to six `LCD_Digit` objects, supports numeric display, character arrays, per-digit colour control, direct segment control, and render delegation.

7. Configuration handling

Configuration is currently compile-time and source-file based:

- Hardware pins, dimensions, and timing constants are in `Games/Tetris/Pins.h` and `Games/Breakout/Pins.h`.
- Arduino local library metadata is in `libraries/PixelGridcore/library.properties`.
- GitHub Actions test triggers and commands are in `.github/workflows/tetris-tests.yml`.
- Optional network credentials and secrets are expected in `Games/Tetris/NetConfig.h`.

8. Error handling

- Bounds checks before writing game pieces and display state.
- Input debounce to avoid noisy hardware input states.
- Host serial packet length validation and flushing/resynchronisation when payloads are invalid.
- Network submission failure returns `false`, with diagnostic `Serial.println` output.
- NTP sync timeout handling before signed network requests.
- HTTP status code checks that reject non-2xx responses.
- Basic JSON field extraction failure handling if `code` cannot be found.

9. Maintainability considerations

Strengths:

- Modular file organisation separates games, input, rendering, pins, shared library code, tests, and tutorials.
- Tetris core rules are concentrated in `TetrisGame`, making selected behaviours testable on a host compiler.
- The local `PixelGridcore` library provides reusable abstractions across sketches.
- CI provides regression protection for selected Tetris logic.

Risks and improvement opportunities:

- `Games/Tetris/Game.h` contains significant implementation inside a header, which can complicate compilation boundaries.
- Tetris runtime state in `Tetris.ino` is extensive and could be refactored into a state-machine module.
- Breakout has no host-side tests.
- Network JSON parsing is minimal and tailored to a single field.
- Hardware constants are duplicated per game rather than centralised in a board profile.
- Vended third-party libraries need explicit update and version policy.

10. Security considerations

10.1 Positive evidence

- Score submission uses HMAC-SHA256 over a canonical message.
- Payload includes a timestamp and nonce, suggesting replay protection on the server side.
- NTP synchronisation is required before signing requests.
- Secrets appear to be expected in `NetConfig.h`, which is absent from the repository and therefore likely not committed.

10.2 Risks

- `WiFiClientSecure::setInsecure()` disables certificate validation for the score submission flow.
- The backend validation implementation is not available for review.
- No `NetConfig.example.h` exists to describe required configuration without exposing secrets.
- Minimal JSON parsing can fail on valid but differently formatted JSON.
- No explicit retry/backoff policy is documented for network failures.

11. Testing design

The host test design uses stubs in `tests/stubs` to allow `Games/Tetris/Game.h` to compile outside Arduino. Tests cover placement bounds, line clears, scoring, level updates, hold mechanics, lock-on-drop, and soft drop delay. The CI workflow compiles with `g++ -std=c++17` and executes the resulting binary.